

Dưới đây là danh sách đầy đủ và cập nhật nhất đến tháng 12/2025 các template YAML cơ bản và nâng cao của Kubernetes mà bất kỳ ai làm DevOps/K8s đều cần nắm chắc. Mình sẽ chia theo nhóm tài nguyên (resource kinds) từ đơn giản đến phức tạp.

## 1. Core Resources (cơ bản nhất)

### Pod

```
YAML

apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
      env:
        - name: ENV
          value: "production"
      resources:
        requests:
          memory: "64Mi"
          cpu: "250m"
        limits:
          memory: "128Mi"
          cpu: "500m"
```



### Deployment (phổ biến nhất)

```
YAML

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
```

```

ports:
  - containerPort: 80
livenessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 15
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 5
  periodSeconds: 5

```

## Service (4 loại chính)

### ClusterIP (default)

YAML

```

apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: ClusterIP

```



### NodePort

YAML

```

apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080

```

### LoadBalancer

YAML

```

apiVersion: v1
kind: Service
metadata:
  name: nginx-lb
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80

```

## Headless Service (dùng với StatefulSet)

YAML

```
apiVersion: v1
kind: Service
metadata:
  name: mysql-headless
  annotations:
    service.alpha.kubernetes.io/tolerate-unready-endpoints: "true"
spec:
  clusterIP: None
  selector:
    app: mysql
  ports:
    - port: 3306
```

## 2. Volume & Storage

### PersistentVolume (PV)

YAML

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-nfs-01
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Recycle
  nfs:
    server: 10.0.0.100
    path: "/exports/data"
```



### PersistentVolumeClaim (PVC)

YAML

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 8Gi
  storageClassName: standard
```

### StorageClass

YAML

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp3
  encrypted: "true"
volumeBindingMode: WaitForFirstConsumer
```

```
reclaimPolicy: Delete
allowVolumeExpansion: true
```

### 3. Workload nâng cao

#### StatefulSet (MySQL, MongoDB, Zookeeper...)

YAML

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: "mysql"
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: password
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql
      volumeClaimTemplates:
        - metadata:
            name: data
          spec:
            accessModes: [ "ReadWriteOnce" ]
            resources:
              requests:
                storage: 10Gi
```



#### DaemonSet

YAML

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd
spec:
  selector:
    matchLabels:
      app: fluentd
  template:
    metadata:
      labels:
        app: fluentd
    spec:
      tolerations:
        - operator: Exists
      containers:
        - name: fluentd
          image: fluent/fluentd-kubernetes-daemonset:v1.16
```

## Job

YAML

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pi-calc
spec:
  template:
    spec:
      containers:
      - name: pi
        image: perl:5.34
        command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(2000)"]
        restartPolicy: Never
      backoffLimit: 4
```

## CronJob

YAML

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: daily-backup
spec:
  schedule: "0 2 * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: my-backup-image:latest
            args: ["--full"]
            restartPolicy: OnFailure
```



## 4. Networking & Ingress

### Ingress (nginx-ingress example)

YAML

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  tls:
  - hosts:
    - app.example.com
    secretName: app-tls
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: app-service
            port:
              number: 80
```

## NetworkPolicy

YAML

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
      - protocol: TCP
        port: 8080
```

## 5. Configuration & Security

### ConfigMap

YAML



```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  database_url: "mysql://db:3306"
  log_level: "debug"
  app.properties: |
    timeout=30
    retries=3
```

### Secret (opaque & docker-registry)

YAML

```
# Opaque Secret
apiVersion: v1
kind: Secret
type: Opaque
metadata:
  name: db-secret
data:
  username: YWRtaW4= # base64 của "admin"
  password: cGFzc3dvcmQ= # base64 của "password"

# Docker registry secret
apiVersion: v1
kind: Secret
type: kubernetes.io/dockerconfigjson
metadata:
  name: docker-registry-secret
data:
  .dockerconfigjson: eyJhdXRob3RocSI6ey... (base64 của docker config)
```

## 6. RBAC (Role-Based Access Control)

### Role & RoleBinding

```
YAML
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]

---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods-binding
  namespace: dev
subjects:
- kind: User
  name: developer-jane
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

## ServiceAccount

```
YAML

apiVersion: v1
kind: ServiceAccount
metadata:
  name: backup-sa
  namespace: prod
```



## 7. Các tài nguyên khác thường dùng

### HorizontalPodAutoscaler (HPA)

```
YAML

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

### PodDisruptionBudget

```
YAML

apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
```

```
name: app-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: critical-app
```

## LimitRange & ResourceQuota

```
YAML

apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
spec:
  limits:
  - type: Container
    default:
      cpu: 500m
      memory: 512Mi
    defaultRequest:
      cpu: 200m
      memory: 256Mi
```

Bạn có thể sao chép toàn bộ các template trên thành một repo Git để dùng làm “Kubernetes YAML Templates” chuẩn cho team.



Nếu bạn cần thêm các template nâng cao hơn như:

- CustomResourceDefinition (CRD) + Operator
- Gateway API (Route, Gateway)
- Istio VirtualService/Gateway
- ArgoCD Application
- Keda ScaledObject
- Cert-Manager Certificate

Hãy nói mình biết, mình sẽ gửi tiếp bộ đầy đủ phần 2!

≡ Add to chat

thêm các dòng chú thích bằng tiếng việt trong các file yaml

Dưới đây là **toàn bộ các template YAML Kubernetes phổ biến nhất** đã được mình **thêm đầy đủ chú thích bằng tiếng Việt** (dễ hiểu cho người Việt Nam, đặc biệt là DevOps, SRE, Backend mới học K8s).

Bạn chỉ cần copy-paste là dùng được ngay, rất tiện để học, dạy team hoặc làm template nội bộ.

## 1. Pod

```
YAML

apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  namespace: default          # Namespace mà Pod chạy, nếu không ghi thì là default
  labels:
    app: nginx                # Label để Service/Deployment chọn Pod
    env: production
spec:
  containers:
  - name: nginx                # Tên container (có thể đặt tùy ý)
    image: nginx:1.27-alpine   # Image Docker sẽ được pull
    imagePullPolicy: IfNotPresent # Chỉ pull nếu chưa có local
    ports:
```

```

- containerPort: 80      # Port mà container lắng nghe bên trong
env:                    # Biến môi trường truyền vào container
- name: ENV
  value: "production"
resources:              # Giới hạn tài nguyên CPU & RAM
  requests:             # Tài nguyên tối thiểu yêu cầu
    memory: "64Mi"
    cpu: "250m"          # 0.25 core
  limits:               # Giới hạn tối đa (hard limit)
    memory: "128Mi"
    cpu: "500m"          # 0.5 core
volumeMounts:          # Gắn volume vào container (nếu có)
- name: data
  mountPath: /usr/share/nginx/html
volumes:                # Khai báo volume (ở đây ví dụ emptyDir)
- name: data
  emptyDir: {}

```

## 2. Deployment (quan trọng nhất)

YAML

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  namespace: production
  labels:
    app: nginx
spec:
  replicas: 5              # Số Pod mong muốn
  strategy:
    type: RollingUpdate    # Chiến lược update
    rollingUpdate:
      maxSurge: 1          # Được tạo thêm tối đa bao nhiêu Pod khi update
      maxUnavailable: 1    # Được phép mất tối đa bao nhiêu Pod khi update
  selector:
    matchLabels:
      app: nginx           # Bắt buộc phải khớp với label của Pod template
  template:                # Blueprint của Pod
    metadata:
      labels:
        app: nginx
        version: v1
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "256Mi"
              cpu: "500m"
          livenessProbe:    # Kiểm tra container còn sống không
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 30  # Chờ 30s mới bắt đầu kiểm tra
            periodSeconds: 10
            failureThreshold: 3
          readinessProbe:    # Kiểm tra container đã sẵn sàng nhận traffic chưa
            httpGet:
              path: /

```

```
port: 80
initialDelaySeconds: 5
periodSeconds: 5
```

### 3. Service (4 loại chính)

ClusterIP (mặc định – chỉ truy cập nội bộ cluster)

YAML

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
  namespace: production
spec:
  type: ClusterIP                # Mặc định nếu không ghi
  selector:
    app: nginx                  # Chọn Pod nào có label này
  ports:
    - protocol: TCP
      port: 80                  # Port mà Service expose ra
      targetPort: 80            # Port của Pod (có thể dùng tên nếu có named port)
```

NodePort (truy cập từ bên ngoài qua NodeIP:NodePort)

YAML



```
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080           # Port cố định trên tất cả node (30000-32767)
```

LoadBalancer (dùng trên cloud AWS/GCP/Azure)

YAML

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-lb
spec:
  type: LoadBalancer           # Cloud sẽ tạo public LB tự động
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
```

Headless Service (dùng cho StatefulSet, trả về trực tiếp IP của Pod)

YAML

```
apiVersion: v1
kind: Service
metadata:
  name: mysql-headless
spec:
  clusterIP: None               # Quan trọng: không cấp ClusterIP
```

```

selector:
  app: mysql
ports:
  - port: 3306
    name: mysql

```

#### 4. PersistentVolume & PersistentVolumeClaim

##### PersistentVolume (PV)

```

YAML

apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-nfs-01
spec:
  capacity:
    storage: 100Gi          # Dung lượng
  accessModes:
    - ReadWriteMany        # Nhiều Pod cùng đọc/ghi
  persistentVolumeReclaimPolicy: Retain # Khi PVC xóa, PV vẫn giữ lại
  nfs:
    server: 10.10.10.100
    path: /k8s-data

```

##### PersistentVolumeClaim (PVC)



```

YAML

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
  namespace: database
spec:
  accessModes:
    - ReadWriteOnce        # Chỉ 1 Pod gắn RW0
  resources:
    requests:
      storage: 50Gi         # Yêu cầu dung lượng
      storageClassName: ""  # Để trống nếu dùng PV static

```

#### 5. StatefulSet (MySQL, Redis, MongoDB...)

```

YAML

apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql-headless # Bắt buộc phải có, trùng tên Headless Service
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          env:
            - name: MYSQL_ROOT_PASSWORD

```

```

      valueFrom:
        secretKeyRef:
          name: mysql-secret
          key: password
    ports:
    - containerPort: 3306
    volumeMounts:
    - name: data
      mountPath: /var/lib/mysql
  volumeClaimTemplates: # Tự động tạo PVC riêng cho từng Pod
- metadata:
  name: data
  spec:
    accessModes: [ "ReadWriteOnce" ]
    storageClassName: fast-ssd
    resources:
      requests:
        storage: 50Gi

```

## 6. ConfigMap & Secret

### ConfigMap

YAML

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: production
data:
  database_url: "mysql://mysql-svc:3306/mydb"
  log_level: "info"
  app.conf: |
    server.port=8080
    timeout=30

```



### Secret (Opaque)

YAML

```

apiVersion: v1
kind: Secret
type: Opaque # Loại secret thông thường
metadata:
  name: db-secret
data:
  username: YWRtaW4= # echo -n "admin" | base64
  password: cGFzc3dvcmQxMjM= # echo -n "password123" | base64

```

## 7. Ingress (nginx-ingress hoặc Traefik)

YAML

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  namespace: production
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$1
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  tls: # Tự động cấp SSL bằng cert-manager
  - hosts:
    - app.example.com
    secretName: app-tls-secret

```

```
rules:
- host: app.example.com
  http:
    paths:
      - path: /api/(.*)
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 80
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
```

## 8. HorizontalPodAutoscaler (HPA)

YAML

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: app-deployment
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70 # Tự động scale khi CPU trung bình > 70%
```

Bạn muốn mình tiếp tục bổ sung phần 2 với các template sau (cũng đã có đầy đủ comment tiếng Việt) không?

- CRD + Operator mẫu
- Gateway API (Kubernetes Gateway)
- Istio VirtualService / Gateway
- Cert-Manager Certificate
- ArgoCD Application
- Prometheus ServiceMonitor
- Keda ScaledObject
- Helm-style values.yaml mẫu

Chỉ cần nói “gửi tiếp phần 2” là mình gửi ngay nhé!

Gửi tiếp phần 2

